

Ansible Training – Beginner to AWS EC2 Automation

Course Objective

The objective of this training is to understand Ansible fundamentals and learn how to automate Linux server configuration and AWS EC2 instance provisioning using Ansible.

By the end of this training, students will be able to:

- Understand Ansible architecture.
 - Install and configure Ansible.
 - Configure a Managed Node.
 - Create and manage an Ansible Inventory.
 - Execute Ansible Ad-hoc Commands.
 - Write basic Ansible Playbooks.
 - Secure sensitive information using Ansible Vault.
 - Provision an AWS EC2 instance using Ansible.
-

Module 1: Introduction to Ansible

What is Ansible?

Ansible is an open-source IT automation and configuration management tool.

Ansible is commonly used for:

- Configuration Management
- Application Deployment
- Server Provisioning
- Infrastructure Automation
- Software Installation
- Patch Management
- Cloud Automation

Why Do We Need Ansible?

Consider an administrator managing three servers.

Without Ansible:

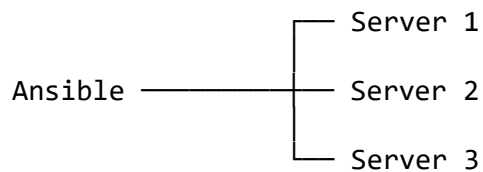
SSH → Server 1 → Install Nginx

SSH → Server 2 → Install Nginx

SSH → Server 3 → Install Nginx

The administrator must manually connect to every server and execute the same commands.

With Ansible:



A single Ansible command or Playbook can configure multiple servers.

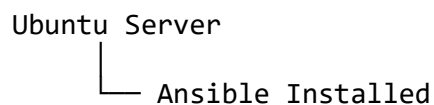
Ansible Architecture

Ansible mainly uses two types of systems.

Control Node

The Control Node is the server where Ansible is installed.

Ansible commands and Playbooks are executed from the Control Node.



Managed Node

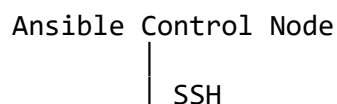
A Managed Node is a server managed by Ansible.

Examples:

AWS EC2 Instance
 Azure Virtual Machine
 Linux Server
 On-Premises Server

How Does Ansible Connect to Managed Nodes?

For Linux Managed Nodes, Ansible normally uses SSH.



▼
Managed Node

Ansible is an **agentless automation tool**.

We do not need to install an Ansible agent on the Managed Node.

However, Python is generally required on Linux Managed Nodes for most Ansible modules.

Important Ansible Components

Component	Description
Control Node	Server where Ansible is installed
Managed Node	Server managed by Ansible
Inventory	Contains Managed Node information
Module	Performs a specific operation
Playbook	YAML file containing automation tasks
Task	Individual automation action
Vault	Encrypts sensitive information

Module 2: Install Ansible on Ubuntu

Ansible should be installed on the Control Node.

Step 1: Update the Package Repository

```
sudo apt update
```

Step 2: Install Ansible

```
sudo apt install ansible -y
```

Step 3: Verify Ansible

```
ansible --version
```

Example output:

```
ansible [core 2.x.x]
python version = 3.x.x
```

The Ansible Control Node is now ready.

Module 3: Setup the Managed Server

For this training, we will use an Ubuntu EC2 instance as the Managed Node.

Managed Node Prerequisites

The Managed Node should have:

- SSH connectivity.
- Python 3 installed.
- A valid SSH user.
- An SSH private key.
- Port 22 allowed from the Ansible Control Node.

Step 1: Verify SSH Connectivity

From the Ansible Control Node:

```
ssh -i /home/ubuntu/SSH-Key/Guvi.pem ubuntu@13.233.192.49
```

If the SSH connection is successful, the server is reachable.

Step 2: Verify Python 3

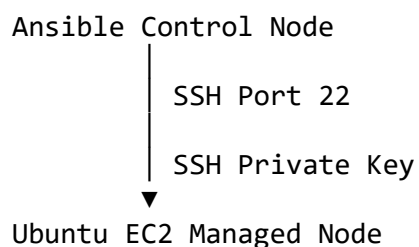
After connecting to the Managed Node:

```
python3 --version
```

If Python 3 is not installed:

```
sudo apt update
sudo apt install python3 -y
```

Managed Node Connectivity Flow



Module 4: Ansible Inventory

What is an Ansible Inventory?

An Ansible Inventory contains information about the Managed Nodes.

The Inventory can define:

- Server IP Address
- Hostname or Alias
- SSH Username
- SSH Private Key
- Server Groups
- Connection Variables

Step 1: Create an Ansible Directory

```
mkdir ansible
cd ansible
```

Step 2: Create an Inventory File

```
nano inventory.ini
```

Add the Managed Node:

```
[ec2]
```

```
ec2-instance ansible_host=13.233.192.49 ansible_user=ubuntu
ansible_private_key_file=/home/ubuntu/SSH-Key/Guvi.pem
```

Why Is localhost Not Added to the Inventory?

Previously, the Inventory contained:

```
localhost ansible_connection=local
ansible_python_interpreter=/usr/bin/python3
```

For this training, we do not need to add localhost to the Inventory.

The Inventory is primarily used to define the **Managed Nodes that Ansible connects to and manages**.

Our Inventory therefore contains only the EC2 Managed Node.

```
inventory.ini
|
▼
EC2 Managed Node
```

For AWS EC2 provisioning, the Playbook will explicitly run on the Control Node using:

```
hosts: localhost
connection: local
```

Therefore, adding localhost to inventory.ini is not required for this lab.

Understanding the Inventory

Host Alias

ec2-instance

This is the name used by Ansible to identify the Managed Node.

```
ansible_host
ansible_host=13.233.192.49
```

Specifies the IP address of the Managed Node.

```
ansible_user
ansible_user=ubuntu
```

Specifies the SSH username.

For Ubuntu EC2:

ubuntu

For Amazon Linux:

```
ec2-user
ansible_private_key_file
ansible_private_key_file=/home/ubuntu/SSH-Key/Guvi.pem
```

Specifies the SSH private key used to connect to the Managed Node.

Inventory Groups

The following configuration creates a group named ec2.

```
[ec2]
```

Multiple servers can be added to the same group.

Example:

```
[ec2]
web-server ansible_host=10.0.1.10 ansible_user=ubuntu
```

```
app-server ansible_host=10.0.1.20 ansible_user=ubuntu
db-server ansible_host=10.0.1.30 ansible_user=ubuntu
```

List Inventory Hosts

```
ansible all -i inventory.ini --list-hosts
```

Expected output:

```
hosts (1):
  ec2-instance
```

Module 5: Ansible Ad-hoc Commands

What is an Ansible Ad-hoc Command?

An Ad-hoc Command is a one-time Ansible command used to perform a quick task on one or more Managed Nodes.

Syntax:

```
ansible <host-or-group> -i <inventory-file> -m <module> -a "<arguments>"
```

Example:

```
ansible ec2 -i inventory.ini -m command -a "uptime"
```

Test Ansible Connectivity

Use the Ansible ping module.

```
ansible ec2 -i inventory.ini -m ping
```

Expected output:

```
ec2-instance | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Important: Ansible Ping Is Not ICMP Ping

The Ansible ping module does not send an ICMP packet.

It verifies:

SSH Connectivity

+

Python Availability

+
Ansible Module Execution

Check Server Uptime

```
ansible ec2 -i inventory.ini -m command -a "uptime"
```

Check Disk Usage

```
ansible ec2 -i inventory.ini -m command -a "df -h"
```

Check Memory

```
ansible ec2 -i inventory.ini -m command -a "free -m"
```

Install Nginx

```
ansible ec2 -i inventory.ini -m apt -a "name=nginx state=present  
update_cache=yes" --become
```

Start Nginx

```
ansible ec2 -i inventory.ini -m service -a "name=nginx state=started" --  
become
```

Enable Nginx

```
ansible ec2 -i inventory.ini -m service -a "name=nginx enabled=yes" --become
```

Module 6: Introduction to Ansible Playbooks

What is an Ansible Playbook?

An Ansible Playbook is a YAML file containing one or more automation tasks.

Example:

```
---  
- name: Install and Configure Nginx  
  hosts: ec2  
  become: true  
  
  tasks:  
    - name: Install Nginx  
      apt:  
        name: nginx  
        state: present
```



```
- name: Start Nginx
  service:
    name: nginx
    state: started
    enabled: true
```

Create the Playbook:

```
nano nginx.yml
```

Run the Playbook:

```
ansible-playbook -i inventory.ini nginx.yml
```

Ad-hoc Command vs Playbook

Ad-hoc Command	Playbook
One-time operation	Reusable automation
Command-line based	YAML based
Suitable for quick tasks	Suitable for multiple tasks
Difficult to maintain	Easy to version control
Quick troubleshooting	Configuration automation

Module 7: Ansible Vault

What is Ansible Vault?

Ansible Vault is used to encrypt sensitive information.

Examples:

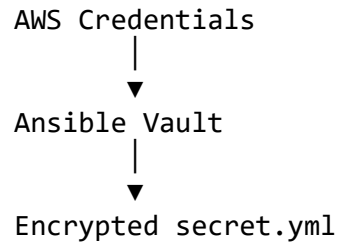
- AWS Access Keys
- Passwords
- API Keys
- Database Credentials
- Tokens

Sensitive information should not be stored as plain text.

Incorrect:

```
aws_access_key: AKIAXXXXXXXXXX
aws_secret_key: xxxxxxxxxxxxxxxxxxxx
```

Recommended flow:



Step 1: Create an Ansible Vault

```
ansible-vault create secret.yml
```

Enter and confirm the Vault password.

Add the AWS credentials:

```
aws_access_key: "YOUR_AWS_ACCESS_KEY"
aws_secret_key: "YOUR_AWS_SECRET_KEY"
```

Save and close the file.

View the Vault File

```
ansible-vault view secret.yml
```

Edit the Vault File

```
ansible-vault edit secret.yml
```

Encrypt an Existing File

```
ansible-vault encrypt secret.yml
```

Decrypt the Vault File

```
ansible-vault decrypt secret.yml
```

Module 8: Create the Vault Password File

Create a Vault password file.

```
nano vault_pass.txt
```

Add the Vault password.

Example:

```
Ansible@123
```

Secure the file:

```
chmod 600 vault_pass.txt
```

Important Security Practice

Never commit the Vault password file to Git.

Add the file to .gitignore.

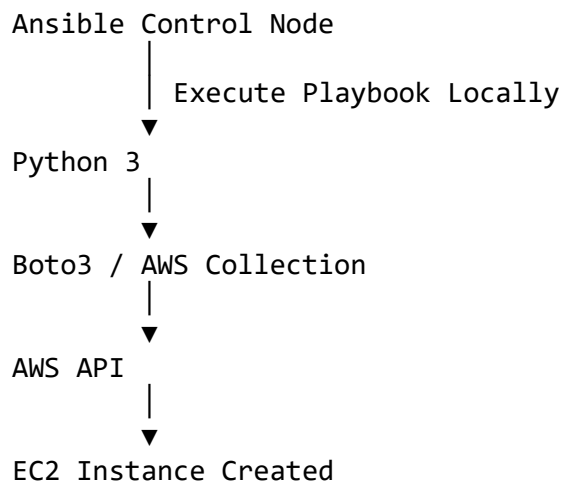
```
vault_pass.txt
```

The secret.yml file can be stored in version control **only while it remains encrypted**, subject to your organization's secret-management policy.

Module 9: AWS EC2 Provisioning Using Ansible

Ansible can provision AWS resources by communicating with AWS APIs.

EC2 Provisioning Architecture



Important Concept

The EC2 provisioning Playbook runs on the **Ansible Control Node**.

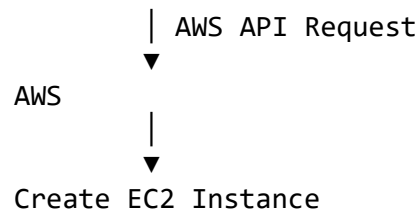
Why?

The EC2 instance does not exist before provisioning.

Therefore, Ansible cannot SSH to the new EC2 instance.

Instead:

```
Ansible Control Node
|
```



This is why the Playbook uses:

```

hosts: localhost
connection: local
  
```

Step 1: Install Python 3 Boto3

Run on the Ansible Control Node:

```
sudo apt install python3-boto3 -y
```

Verify Boto3:

```
python3 -c "import boto3; print(boto3.__version__)"
```

Step 2: Install the AWS Ansible Collection

```
ansible-galaxy collection install amazon.aws
```

Verify the collection:

```
ansible-galaxy collection list
```

Module 10: Create the EC2 Provisioning Playbook

Create a Playbook directory.

```
mkdir Playbook
cd Playbook
```

Create the Playbook:

```
nano ec2_create.yml
```

Add the following configuration:

```

---
- name: Create AWS EC2 Instance
  hosts: localhost
  connection: local
  gather_facts: false
  
```

```
vars_files:
  - secret.yml

tasks:

  - name: Launch EC2 Instance
    amazon.aws.ec2_instance:
      name: "Ansible-EC2"
      key_name: "Guvi"
      instance_type: "t2.micro"
      image_id: "ami-xxxxxxxxxxxxxxxxxxx"
      region: "ap-south-1"

      aws_access_key: "{{ aws_access_key }}"
      aws_secret_key: "{{ aws_secret_key }}"

      wait: true
      state: running
```

Understanding the EC2 Playbook

hosts: localhost

hosts: localhost

The Playbook targets the Ansible Control Node.

The AWS provisioning task is initiated from the Control Node.

connection: local

connection: local

Ansible executes the task directly on the Control Node.

Ansible does not use SSH to connect to localhost.

gather_facts: false

gather_facts: false

Ansible Fact Gathering is disabled.

We are provisioning an AWS resource and do not require Control Node system information.

vars_files

vars_files:

- secret.yml

Loads variables from the encrypted Ansible Vault file.

```
amazon.aws.ec2_instance  
amazon.aws.ec2_instance:
```

This module manages AWS EC2 instances.

AWS Credentials

```
aws_access_key: "{{ aws_access_key }}"  
aws_secret_key: "{{ aws_secret_key }}"
```

The values are loaded from:

```
secret.yml
```

Ansible Vault decrypts the file during Playbook execution.

Module 11: Run the EC2 Provisioning Playbook

Check Playbook Syntax

```
ansible-playbook ec2_create.yml --syntax-check
```

If the Playbook loads an encrypted secret.yml, provide the Vault password:

```
ansible-playbook ec2_create.yml --syntax-check --vault-password-file  
vault_pass.txt
```

Run the Playbook

```
ansible-playbook ec2_create.yml --vault-password-file vault_pass.txt
```

Do We Need -i inventory.ini?

No.

For this EC2 provisioning Playbook:

```
hosts: localhost  
connection: local
```

The Playbook runs locally on the Ansible Control Node.

Therefore:

```
ansible-playbook ec2_create.yml --vault-password-file vault_pass.txt
```

is sufficient.

The inventory.ini file is used when managing the existing EC2 Managed Node.

Example:

```
ansible-playbook -i inventory.ini nginx.yml
```

Configuration Management vs Cloud Provisioning

Configuration Management

Used to configure an existing server.

Ansible Control Node

↓ SSH

Existing EC2 Instance

↓

Install Nginx

Inventory is required:

[ec2]

```
ec2-instance ansible_host=13.233.192.49 ansible_user=ubuntu
ansible_private_key_file=/home/ubuntu/SSH-Key/Guvi.pem
```

Run:

```
ansible-playbook -i inventory.ini nginx.yml
```

Cloud Provisioning

Used to create AWS resources.

Ansible Control Node

↓ AWS API

AWS

↓

Create EC2 Instance

Playbook:

```
hosts: localhost
connection: local
```

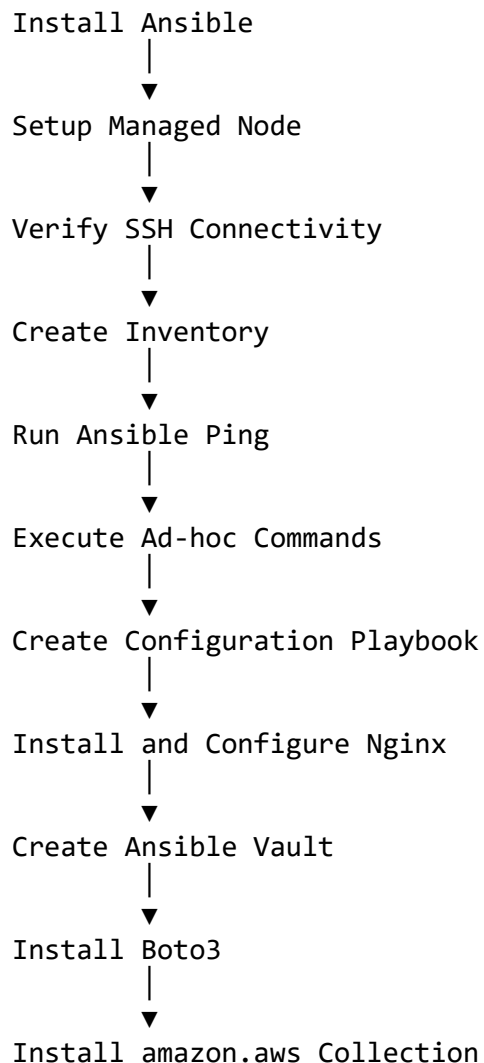
Run:

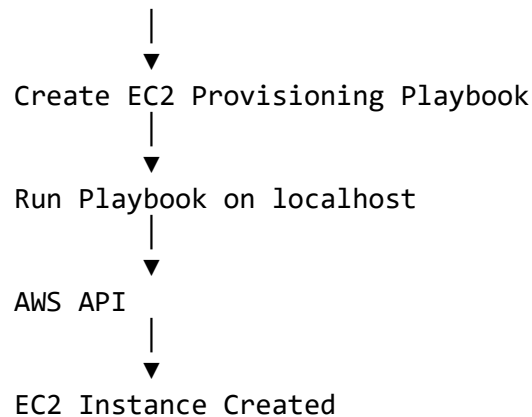
```
ansible-playbook ec2_create.yml --vault-password-file vault_pass.txt
```

Key Difference

Configuration Management	Cloud Provisioning
Manages existing servers	Creates cloud resources
Uses SSH	Uses AWS APIs
Uses Inventory	Can run on localhost
Managed Node required	Managed Node not required initially
Example: Install Nginx	Example: Create EC2

Complete Ansible Training Workflow





Student Hands-On Task

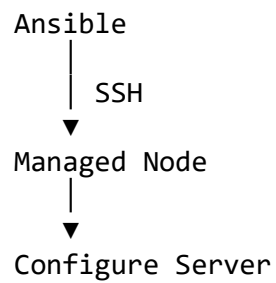
Students should complete the following activities:

1. Install Ansible on an Ubuntu Control Node.
2. Create an Ubuntu EC2 Managed Node.
3. Verify SSH connectivity from the Control Node.
4. Create an Ansible Inventory.
5. Add the EC2 Managed Node to the Inventory.
6. Test connectivity using the Ansible ping module.
7. Check server uptime using an Ad-hoc Command.
8. Check disk usage using an Ad-hoc Command.
9. Install Nginx using an Ad-hoc Command.
10. Create an Nginx configuration Playbook.
11. Execute the Nginx Playbook against the Managed Node.
12. Create an Ansible Vault.
13. Store AWS credentials in the Vault.
14. Create a Vault password file.
15. Install Python Boto3.
16. Install the `amazon.aws` collection.
17. Create an EC2 provisioning Playbook.
18. Execute the EC2 provisioning Playbook locally.
19. Verify the new EC2 instance in AWS.

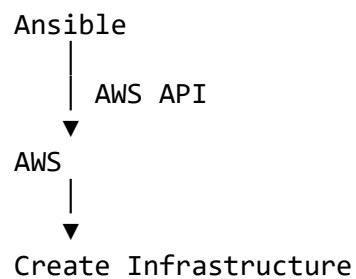
Final Concept

Ansible can perform two different types of automation in this lab.

Manage Existing Infrastructure



Provision New Infrastructure



Inventory answers: “Which existing machines should Ansible manage?”

hosts: localhost with connection: local answers: “Run this automation from the Ansible Control Node itself.”